

ESP32 Bluetooth

Author: Brendan Klem

Goal: Implement Bluetooth communication on the ESP32 for wireless sensor data transmission and develop a user-friendly application that abstracts the underlying code, allowing users to connect to the device and record data without requiring any programming knowledge.

Materials List

- M5Stick S3 Mini IoT Dev Kit
- Watch Accessory Kit for M5Stick Series
- M5StickC Proto Hat
- Resistors: 10k Ω , 1M Ω
- 4 Wires
- 4 Male-Female Jumper Wires
- 3x2 pin male-male connector
- Basic Soldering Kit

General Notes

The project uses an M5Stick S3, a compact development board based on the ESP32-S3 microcontroller. The ESP32-S3 is the central controller for the wearable sensor system.

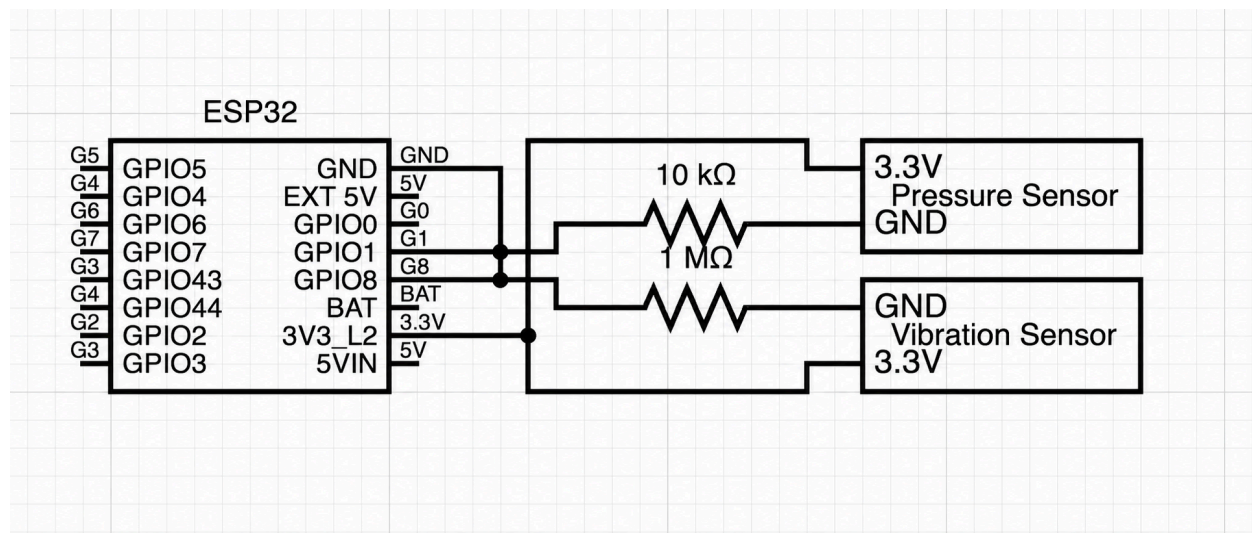
Two external analog sensors are connected to the ESP32:

- FlexiForce pressure sensor for measuring applied force.
- Piezoelectric vibration sensor for detecting movement and impacts.

The analog voltages from these sensors are converted into digital values using the ESP32's 12-bit ADC, producing readings from 0 to 4095. The ESP32 also reads data from its onboard IMU, which provides three-axis gyroscope and accelerometer measurements.

Circuit Layout:

Here is the circuit layout I used to connect both the pressure sensor and the vibration sensor to the ESP32. The IMU is connected internally.



Arduino Code

Library Initialization:

The M5Unified library was used to make communication with the M5Stick S3 hardware much simpler, reducing the amount of code required to access the display, IMU, and button.

Display Interface:

The software includes both graphical and numerical display modes to provide multiple ways of viewing sensor data. The graphing functionality was adapted from an official M5Stack example and integrated with the custom sensor readings and a screen rotation system.

Sensor Acquisition:

The ESP32 continuously samples the pressure sensor, vibration sensor, and onboard IMU. Analog sensor signals are converted to digital values using the ESP32's built-in 12-bit ADC for processing and display.

Data Logging:

Sensor readings are formatted into separate values and transmitted to a computer, allowing a Python program to automatically record the data for later analysis.

Program Operation:

The main loop repeatedly reads the sensors, updates the display, transmits the latest measurements, and checks for user input.

```

#include <M5Unified.h>

const int pressurePin = 1;
const int vibrationPin = 8;

int screenMode = 0;

struct rect_t {
    int32_t x;
    int32_t y;
    int32_t w;
    int32_t h;
};

static auto &dsp = M5.Display;
static rect_t rect_graph_area;
static int prev_xpos[9];

static constexpr const uint32_t color_tbl[9] = {
    0xFF0000u, 0xCCCC00u, 0xCC00FFu,
    0xFFCC00u, 0x00FF00u, 0x0088FFu,
    0xFF00CCu, 0x00FFCCu, 0x0000FFu
};

static constexpr const float coefficient_tbl[3] = {
    0.5f,
    1.0f / 256.0f,
    1.0f / 1024.0f
};

void switchScreenMode() {
    screenMode++;

    if (screenMode > 7) {
        screenMode = 0;
    }

    memset(prev_xpos, 0, sizeof(prev_xpos));

    int rotation = screenMode / 2;
    M5.Lcd.setRotation(rotation);

```

```

    M5.Lcd.clear();
}

void waitWithButtonCheck(int waitTime) {
    unsigned long startTime = millis();

    while (millis() - startTime < waitTime) {
        M5.update();

        if (M5.BtnA.wasPressed()) {
            switchScreenMode();
            return;
        }

        delay(10);
    }
}

void drawBar(int32_t ox, int32_t oy, int32_t nx, int32_t px, int32_t h, uint32_t
color) {
    uint32_t bgcolor = (color >> 3) & 0x1F1F1Fu;

    if (px && ((nx < 0) != (px < 0))) {
        dsp.fillRect(ox, oy, px, h, bgcolor);
        px = 0;
    }

    if (px != nx) {
        if ((nx > px) != (nx < 0)) {
            bgcolor = color;
        }

        dsp.setColor(bgcolor);
        dsp.fillRect(nx + ox, oy, px - nx, h);
    }
}

void drawGraph(const rect_t& r, const m5::imu_data_t& data) {
    int ox = (r.x + r.w) >> 1;
    int oy = r.y;
    int h = r.h / 9;

```

```

dsp.startWrite();

for (int index = 0; index < 9; index++) {
    float coe = coefficient_tbl[index / 3] * r.w;
    float xval = data.value[index] * coe;

    int nx = xval;
    int px = prev_xpos[index];

    if (nx != px) {
        prev_xpos[index] = nx;
    }

    drawBar(ox, oy + h * index, nx, px, h - 1, color_tbl[index]);
}

dsp.endWrite();
}

void drawGraphSensorValues(int pressure, int vibration) {
    M5.Lcd.setTextColor(TFT_WHITE, TFT_BLACK);
    M5.Lcd.setTextSize(1);
    M5.Lcd.setFont(1);

    M5.Lcd.fillRect(0, M5.Lcd.height() - 20, M5.Lcd.width(), 20, TFT_BLACK);

    M5.Lcd.setCursor(5, M5.Lcd.height() - 16);
    M5.Lcd.print("P:");
    M5.Lcd.print(pressure);

    M5.Lcd.setCursor(M5.Lcd.width() / 2, M5.Lcd.height() - 16);
    M5.Lcd.print("V:");
    M5.Lcd.print(vibration);
}

void drawValues(const m5::imu_data_t& imu, int pressure, int vibration) {
    M5.Lcd.clear();
    M5.Lcd.setTextColor(TFT_WHITE, TFT_BLACK);
    M5.Lcd.setTextSize(1);

```

```
int rotation = screenMode / 2;

if (rotation == 0 || rotation == 2) {
    M5.Lcd.setTextFont(&fonts::FreeMonoBold9pt7b);

    M5.Lcd.setCursor(0, 5);
    M5.Lcd.print("Time: ");
    M5.Lcd.println(millis());

    M5.Lcd.setCursor(0, 40);
    M5.Lcd.print("P: ");
    M5.Lcd.println(pressure);
    M5.Lcd.print("V: ");
    M5.Lcd.println(vibration);

    M5.Lcd.setCursor(0, 90);
    M5.Lcd.printf("GX: %.2f\n", imu.gyro.x);
    M5.Lcd.printf("GY: %.2f\n", imu.gyro.y);
    M5.Lcd.printf("GZ: %.2f\n", imu.gyro.z);

    M5.Lcd.setCursor(0, 165);
    M5.Lcd.printf("AX: %.2f\n", imu.accel.x);
    M5.Lcd.printf("AY: %.2f\n", imu.accel.y);
    M5.Lcd.printf("AZ: %.2f\n", imu.accel.z);
}
else {
    M5.Lcd.setTextFont(1);
    M5.Lcd.setTextSize(1);

    M5.Lcd.setCursor(0, 5);
    M5.Lcd.printf("T:%lu", millis());

    M5.Lcd.setCursor(0, 20);
    M5.Lcd.printf("P:%d    V:%d", pressure, vibration);

    M5.Lcd.setCursor(0, 55);
    M5.Lcd.print("GYRO");

    M5.Lcd.setCursor(0, 70);
    M5.Lcd.printf("GX:%6.2f", imu.gyro.x);
```

```

M5.Lcd.setCursor(0, 85);
M5.Lcd.printf("GY:%6.2f", imu.gyro.y);

M5.Lcd.setCursor(0, 100);
M5.Lcd.printf("GZ:%6.2f", imu.gyro.z);

M5.Lcd.setCursor(115, 55);
M5.Lcd.print("ACCEL");

M5.Lcd.setCursor(115, 70);
M5.Lcd.printf("AX:%6.2f", imu.accel.x);

M5.Lcd.setCursor(115, 85);
M5.Lcd.printf("AY:%6.2f", imu.accel.y);

M5.Lcd.setCursor(115, 100);
M5.Lcd.printf("AZ:%6.2f", imu.accel.z);
}
}

void setup() {
  auto cfg = M5.config();
  M5.begin(cfg);

  USBSerial.begin(115200);
  delay(1000);
  USBSerial.println("M5 normal mode started");

  M5.Lcd.setRotation(0);
  M5.Lcd.clear();
}

void loop() {
  M5.update();

  if (M5.BtnA.wasPressed()) {
    switchScreenMode();
  }

  int pressure = analogRead(pressurePin);
  int vibration = analogRead(vibrationPin);

```

```
if (pressure < 20) {
    pressure = 0;
}

if (vibration < 20) {
    vibration = 0;
}

M5.Imu.update();
auto imu = M5.Imu.getImuData();

USBSerial.printf("%lu,%d,%d,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f\n",
                 millis(),
                 pressure,
                 vibration,
                 imu.gyro.x,
                 imu.gyro.y,
                 imu.gyro.z,
                 imu.accel.x,
                 imu.accel.y,
                 imu.accel.z);

int rotation = screenMode / 2;
bool graphMode = screenMode % 2;

M5.Lcd.setRotation(rotation);

if (graphMode) {
    rect_graph_area = {0, 0, M5.Lcd.width(), M5.Lcd.height() - 20};
    drawGraph(rect_graph_area, imu);
    drawGraphSensorValues(pressure, vibration);
}
else {
    drawValues(imu, pressure, vibration);
}

waitWithButtonCheck(100);
}
```

Installing ESP32 Library

1. Install board support

In **Arduino IDE**:

1. Go to **Arduino IDE** → **Settings**
2. Find **Additional Board Manager URLs**
3. Add this URL:

https://espressif.github.io/arduino-esp32/package_esp32_index.json

4. Go to **Tools** → **Board** → **Boards Manager**
5. Search **esp32**
6. Install **esp32 by Espressif Systems**

2. Install bleak

```
python3 -m pip install bleak
```

Python Code to Gather Bluetooth Data

- The Python program uses the Bleak library to establish a Bluetooth Low Energy connection with the M5Stick S3. The program automatically scans for the device, connects to it, and gathers incoming sensor data.
- Once connected, the program continuously receives sensor data transmitted by the ESP32. The received data is separated into individual values and written to a timestamped CSV file stored on the user's desktop.
- The software continuously searches for the M5Stick S3 if it is not detected or becomes disconnected. This allows the program to automatically reconnect without requiring the user to restart the application, making the data collection process more reliable and easier to use.

```

import asyncio
import csv
from datetime import datetime
from pathlib import Path
from bleak import BleakScanner, BleakClient

DEVICE_NAME = "M5-Sensor"
SENSOR_UUID = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

desktop = Path.home() / "Desktop"
timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
filename = desktop / f"m5_sensor_data_{timestamp}.csv"

print(f"Saving data to: {filename}")
print("Searching for M5-Sensor...")

file = open(filename, "w", newline="")
writer = csv.writer(file)
writer.writerow(["time_ms", "pressure", "vibration", "
", "gx", "gy", "gz", "
", "ax", "ay", "az"])

buffer = ""

def received(sender, data):
    global buffer

    buffer += data.decode("utf-8", errors="ignore")

    while "\n" in buffer:
        line, buffer = buffer.split("\n", 1)

        parts = line.strip().split(",")

        if len(parts) == 9:

writer.writerow([parts[0], parts[1], parts[2], "", parts[3], parts[4], parts[5], "
", parts[6], parts[7], parts[8]])
            file.flush()
            print(parts)
    async def main():
        while True:
            device = await BleakScanner.find_device_by_name(DEVICE_NAME)
            if device is None:

```

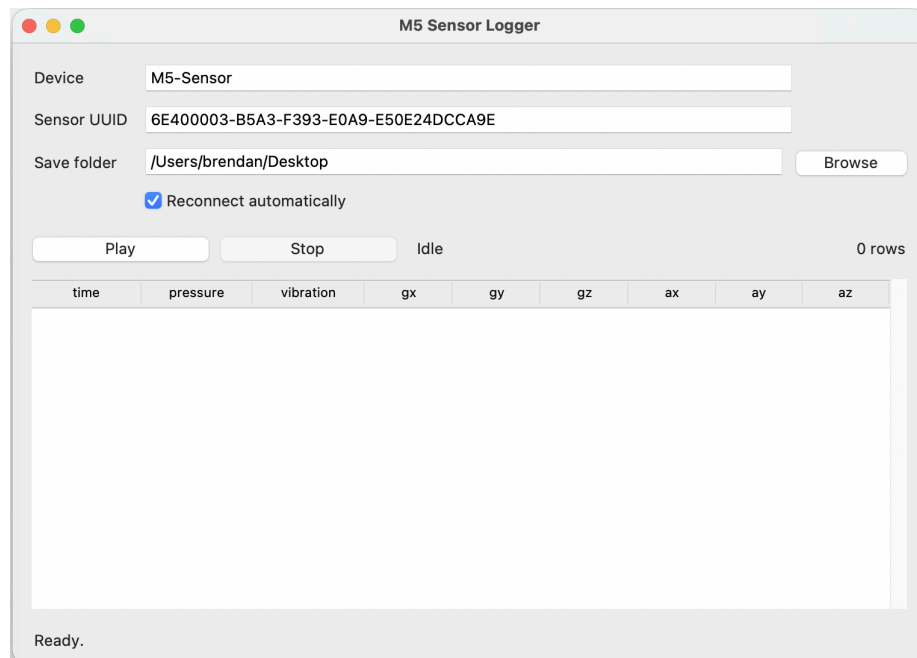
```

        print("M5-Sensor not found. Retrying...")
        await asyncio.sleep(1)
        continue
    print("Connected. Logging data. Press Ctrl+C to stop.")
    async with BleakClient(device) as client:
        await client.start_notify(SENSOR_UUID, received)
        while client.is_connected:
            await asyncio.sleep(1)
        print("Disconnected. Reconnecting...")
try:
    asyncio.run(main())
except KeyboardInterrupt:
    print("\nStopped logging.")
    file.close()

```

Sensor Logger App

A standalone desktop application was developed to eliminate the need for users to interact with the Python source code or an integrated development environment (IDE). This simplifies the testing process and allows sensor data to be collected with minimal user interaction.



Sample Output

When you open the file in the folder you chose to save the data, you will see:

time_ms	pressure	vibration	gx	gy	gz	ax	ay	az
844423	0	0	1.10	1.91	-1.62	0.21	0.40	0.89
844554	0	0	1.29	0.88	-1.08	0.20	0.40	0.90
844685	0	0	3.36	5.09	-2.97	0.21	0.39	0.89
844816	0	0	4.70	27.37	-1.62	0.16	0.45	0.89
844947	0	0	-2.19	14.79	-7.42	0.12	0.43	0.90
845078	0	0	0.67	17.29	-10.41	0.15	0.41	0.90
845209	0	0	-5.06	1.24	-16.46	0.02	0.56	0.89
845340	0	0	-15.99	-2.60	-24.63	0.00	0.33	0.93
845471	0	0	15.38	1.00	4.48	0.02	0.45	0.90
845602	0	0	-1.95	0.63	-1.14	0.00	0.41	0.90
845733	0	0	13.43	-1.38	5.27	0.02	0.39	0.91
845864	0	0	2.69	-3.09	2.16	0.02	0.46	0.88
845995	0	0	-0.91	-4.31	-3.82	0.04	0.44	0.91
846126	0	0	-0.61	-1.87	3.93	0.04	0.39	0.90